CriticRAG: Project and Workflow Guide

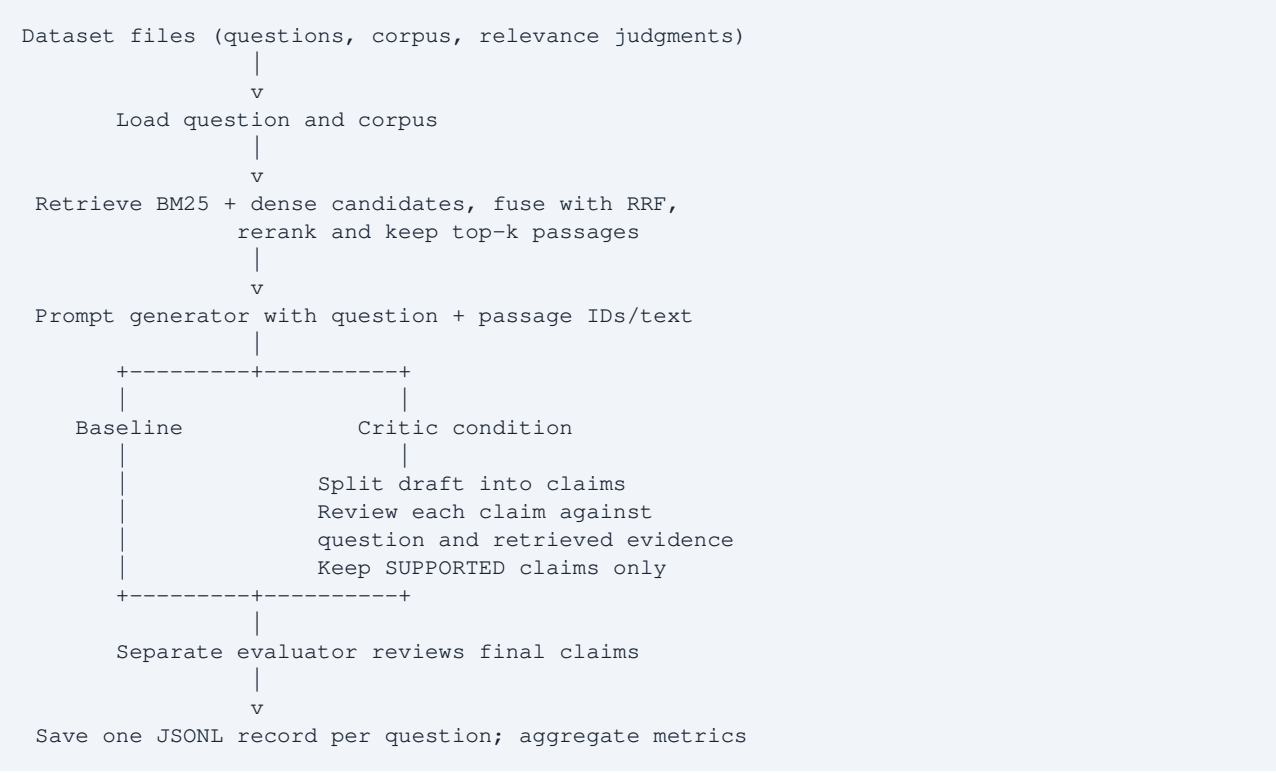
Purpose. This guide explains the files and execution flow in this repository, how a question travels through retrieval and generation, how the current metrics are calculated, and what to adapt for another dataset or question set. It reflects the code and artifacts currently in this checkout (20 October 2026).

1. What the project does

CriticRAG is research code for comparing ordinary retrieval-augmented generation (RAG) with a version that asks a second model to review answer claims against retrieved passages. Its current benchmark is BEIR FiQA, a finance question and passage retrieval dataset. Both baseline and critic runners now use hybrid BM25 and dense retrieval, RRF fusion, and a cross-encoder reranker.

The data distinctions matter: FiQA qrels say which passages are relevant to each question. They are not reference answers and do not label individual answer claims. Claim faithfulness and citation scores produced by the model evaluator are automatic estimates; use independent human annotations before treating them as research findings.

2. How a question moves through the system



Step-by-step details

1. **Load.** `ingestion/fiqa.py` reads FiQA `corpus.jsonl`, `queries.jsonl`, and a selected `qrels/{split}.tsv`. Every question record has a question string and relevant passage IDs as `ground_truth_chunks`. 2. **Retrieve.** `retrieval/bm25.py` ranks passages lexically, while `retrieval/dense_search.py` ranks them by sentence-transformer embeddings. `retrieval/hybrid.py` fuses both candidate lists with reciprocal rank fusion (RRF). The default `BAAI/bge-reranker-base` cross-encoder reranks the fused candidates before the top `k` are sent to generation. Both experiment runners share this path. 3. **Generate.** `generation/generator.py` sends the question and retrieved chunks to a configured Ollama or OpenAI-compatible chat endpoint. The prompt requests answers grounded only in evidence

and citations in the form `[chunk_id]`. 4. **Optional critic.** In `--condition critic`, `generation/claim_extractor.py` splits the draft into sentence-sized candidate claims. `critic/verifier.py` asks the critic to label each claim `SUPPORTED`, `UNSUPPORTED`, or `CONTRADICTED` and checks the JSON and evidence IDs. `critic/correction.py` retains supported claims and removes the others; suggested corrections are not inserted into the answer. 5. **Evaluate.** The runner uses a separate evaluator pass on the final answer's claims and writes verdicts, supporting IDs, and rationale. This is still automated model evaluation, not human annotation. In baseline mode, the answer is unchanged but the same evaluator pass provides comparable claim labels. 6. **Save.** `experiments/run_rag.py` appends a JSON object per question to a new JSONL file under `results/raw/`. It records retrieved IDs, qrel gold IDs, answer, claims, model names, latencies, responses, and status. 7. **Score.** `experiments/calculate_metrics.py` aggregates retrieval recall/hit rate, claim hallucination rates, and citation faithfulness from saved per-question records.

3. Repository file guide

Main project files

- `README.md` — setup, current commands, dataset scope, limitations, and experiment status.
- `pyproject.toml` — package metadata and dependencies. Base dependency is PyYAML; dense retrieval extras are `sentence-transformers` and `torch`; test extra is pytest.
- `.env.example` — example environment variable names; it is not loaded automatically.
- `.gitignore` — files excluded from version control.

Configuration (`configs/`)

- `base.yaml` — seed, default dataset/split, retrieval settings, and critic flags.
- `dataset.yaml` — FiQA paths, split, retrieval depths, and explicitly unavailable labels.
- `models.yaml` — inference backend, endpoint, generator, critic, and evaluator model names.
- `experiments/baseline_rag.yaml` — baseline condition metadata.
- `experiments/same_model_critic.yaml` — same-model critic condition metadata.
- `experiments/cross_model_critic.yaml` — cross-model critic condition metadata.

Ingestion (`ingestion/`)

- `__init__.py` — package marker.
- `fiqa.py` — downloads/checksums BEIR FiQA and normalizes questions plus qrels into records.
- `chunker.py` — deterministic paragraph-aware character chunking with offsets; available for document ingestion, not used by the current FiQA benchmark path (FiQA passages are already units).

Retrieval (`retrieval/`)

- `__init__.py` — package marker.
- `bm25.py` — local lexical index, tokenization, scoring, and ranked passage IDs.
- `dense_search.py` — sentence-transformers embeddings and normalized-vector dot-product ranking, with a persistent fingerprinted passage-vector cache.
- `rrf.py` — reciprocal rank fusion for combining ranked lists.
- `hybrid.py` — combines BM25 and dense rankings with RRF and optionally sends fused candidates to a reranker.
- `reranker.py` — cross-encoder reranks the RRF candidate set against each question.

Answer generation (`generation/`)

- `__init__.py` — package marker.
- `generator.py` — Ollama and OpenAI-compatible API adapters, backend selection, and evidence-grounded answer prompt.
- `claim_extractor.py` — sentence-based claim candidates and bracket citation extraction. This is a simple heuristic, not semantic atomic-claim detection.

Critic (`critic/`)

- `__init__.py` — package marker.
- `verifier.py` — constructs evidence-bounded prompts, parses and validates verdict JSON and chunk IDs, and runs per-claim reviews.
- `evidence_checker.py` — flags citations that do not match retrieved chunk IDs.
- `correction.py` — keeps supported claims, drops other claims, and returns corrections separately without inventing replacements.

Pipelines (`pipeline/`)

- `__init__.py` — package marker.
- `baseline.py` — small callable baseline generation wrapper.
- `critic_rag.py` — callable generator → claim review → conservative filtering flow.
- `retrieval_cache.py` — in-memory retrieval cache keyed by question and configuration fingerprint; not persistent and not used by the current CLI runner.

Evaluation (`evaluation/`)

- `__init__.py` — package marker.
- `retrieval_metrics.py` — evidence-set Recall@k and query-level hit rate.
- `claim_metrics.py` — claim counts, hallucination rate, supported fraction, and any-unsupported indicator.
- `citation_metrics.py` — citation support and completeness helpers; aggregate citation faithfulness requires a supported verdict and overlap with evaluator supporting IDs.
- `aggregate.py` — calculates aggregate metrics from JSONL record objects.
- `statistical_tests.py` — exact paired binary McNemar calculation helper; not wired into the command line flow.

Experiment commands (`experiments/`)

- `__init__.py` — package marker.
- `run_baseline.py` — actual FiQA BM25 retrieval evaluation command; can optionally download FiQA.
- `run_rag.py` — actual FiQA BM25 plus generation runner for baseline/critic conditions.
- `calculate_metrics.py` — reads JSONL or a report with `per_example` rows and writes aggregate JSON.
- `run_critic.py` — currently a configuration/help stub; full critic execution is through `run_rag.py` or the Python pipeline.
- `model_comparison.py` — validates/prints supplied model IDs only; does not run experiments.
- `ablations.py` — validates/prints retrieval depth/regeneration settings only; does not run experiments.
- `experiment_registry.yaml` — current experiment readiness/status record.

Data and results

- `data/raw/fiqa.zip` — downloaded FiQA source archive.

- `data/raw/fiqa/fiqa/corpus.jsonl` — passage corpus.
- `data/raw/fiqa/fiqa/queries.jsonl` — questions.
- `data/raw/fiqa/fiqa/qrels/train.tsv`, `dev.tsv`, `test.tsv` — relevance judgments by split.
- `data/evaluation/annotation_guidelines.md` — instructions for independent claim-level labeling.
- `results/raw/*.jsonl` — per-question model output records. Filenames include baseline/critic, split, or smoke-test variants; one empty baseline JSONL exists and should not be used for scoring.
- `results/metrics/*.json` — saved retrieval/aggregate summaries. These are outputs from prior runs; rerunning commands may overwrite the default metric output path.
- `criticrag.egg-info/*` — package installation metadata generated by setuptools; not application logic.
- `__pycache__/*` — Python bytecode generated at runtime; safe to ignore when reading the source project.

4. Run the current FiQA workflow

From the project root, install the package:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e .
```

Run hybrid retrieval-only evaluation for FiQA test questions (BM25 + dense + RRF + cross-encoder):

```
python -m experiments.run_baseline --split test --k 10 --output results/metrics/fiqa_retrieval_k
python -m experiments.calculate_metrics --input results/metrics/fiqa_retrieval_k10.json --k 3 5
```

`run_baseline` expects files at `data/raw/fiqa/fiqa` by default. Add `--download` if they are absent. To test only a small subset, add `--limit 10`. The first run computes and stores passage vectors under `.cache/criticrag`; later runs reuse them. Add `--reranker-model none` to disable only the cross-encoder stage.

For generated answers, configure models in `configs/models.yaml` (the default backend is local Ollama). Both conditions use the same hybrid retrieval settings:

```
python -m experiments.run_rag --condition baseline --limit 5 --k 10
python -m experiments.run_rag --condition critic --limit 5 --k 10
```

Use `--limit 0` to run all questions in a split. This entails model inference and may take significant time. The runner refuses to overwrite an output file; use the generated path printed at completion.

Then score that output:

```
python -m experiments.calculate_metrics --input results/raw/critic_fiqa_test_TIMESTAMP.jsonl --k
```

Replace the example timestamp with the exact filename printed by your run. Use distinct output filenames to preserve runs.

5. Add another dataset

The current CLI code imports `ingestion.fiqa.load_records` directly, so editing only `configs/dataset.yaml` does not switch datasets. Implement a dataset adapter and update the runner imports or add a dataset-specific runner.

1. **Prepare a corpus.** Convert documents to UTF-8 JSONL. Each line should have a stable unique `_id` and text; an optional `title` is concatenated with the text by current runners. 2. **Prepare questions.** Each question needs a stable ID and text. Use one JSON object per line with fields compatible with the loader, or write a dedicated adapter. 3. **Prepare relevance labels.** For retrieval metrics, provide a qrels TSV with a header followed by `query_id<TAB>document_id<TAB>relevance`. Positive relevance values become gold passage IDs in `load_records`. 4. **Map IDs consistently.** Question IDs in qrels must match the query file and document IDs must match corpus `_id` values. Current code silently skips qrels rows whose query or document is missing, so validate counts and ID matches when adding a dataset. 5. **Keep labels separate.** If you want answer correctness, answerability, question category, or human claim faithfulness metrics, add those annotations as separate fields and define their protocol. FiQA does not provide these labels. 6. **Wire the adapter.** A practical route is to implement `ingestion/<dataset>.py` with a `load_records(root, split)` function returning records like:

```
{"id":"sample_test_q1","question":"Your question?","ground_truth_chunks":["passage_7"],"ground_t
```

Then adapt `experiments/run_baseline.py` and `experiments/run_rag.py` to load that corpus and records, or add a `--dataset` switch that selects the adapter. Keep all corpus/qrel document IDs unchanged through retrieval and output.

7. **Check the dataset license and schema.** Record source, version, split, preprocessing, qrel threshold, and any exclusions in configuration and the experiment output before comparing metrics.

6. Change or add questions

For a retrieval-only custom question set, create a queries JSONL and qrels TSV under your dataset folder, then use a loader that pairs each question with its qrel passage IDs. Questions without qrels can still be retrieved, but Recall@k and hit rate are undefined for them.

For generation experiments, records need at least `id`, `question`, and `ground_truth_chunks`. The RAG runner uses `question` for the prompt and `id` for the saved row. It does not use `ground_truth_answer` to score answer correctness. To change the current FiQA questions, replace/add query and qrel records in a separate dataset directory and point `--data` at that directory; do not overwrite the source benchmark if you need reproducibility.

Pair baseline and critic comparisons on exactly the same question IDs, split, retrieval settings, and retrieved evidence. Save to separate JSONL paths, and ensure each output has one row per same question ID. The current CLI processes records in their loaded order, so changing the dataset requires comparing IDs, not just row positions.

7. What the metrics mean

- **Evidence-set Recall@k:** number of relevant gold passages appearing in the top k, divided by the number of relevant passages for that query. Aggregate score is the mean across eligible queries.
- **Query-level hit rate@k:** 1 if at least one relevant passage is in the top k, otherwise 0; aggregate is the mean across eligible queries.
- **Hallucination rate (automated proxy):** (`UNSUPPORTED` + `CONTRADICTED`) divided by claims with one of the three valid verdicts. No evaluable claims means undefined (`null`), not zero.
- **Citation faithfulness (automated proxy):** cited claims that are judged `SUPPORTED` and cite at least one evaluator-identified supporting passage, divided by all cited claims. No cited claims means undefined.
- **Answer correctness:** not currently measured. The repository has no verified FiQA reference answers for this purpose.

The current retrieval calculator omits queries when fewer than requested k passages were saved, and

reports the eligible query count. Retrieve at the largest k you plan to report. Metric estimates from an LLM evaluator should be validated against blinded human labels; use two annotators and adjudication as described in `data/evaluation/annotation_guidelines.md`.

8. Current implementation limits

- The checked-in benchmark and CLI are FiQA-specific; configuration files alone do not make the runner dataset-agnostic.
- Both CLI runners use BM25 + dense + RRF + cross-encoder reranking by default. Previously saved metrics remain BM25-only until hybrid retrieval is run and new metrics are saved.
- The critic is a claim filter; it does not rewrite unsupported claims in the runnable CLI flow. Regeneration is not implemented as an experiment condition.
- `run_critic.py`, `model_comparison.py`, and `ablations.py` do not run full experiments; see `experiment_registry.yaml` for status.
- Automated evaluator verdicts can be biased or inconsistent, especially when evaluator and generator use the same model. Record model names and validate a human-labeled sample.
- Preserve raw records and write each metric summary to a unique path so comparisons remain auditable.